CHAPTER – 4

ANALYZING FUNCTIONAL DEPENDENCIES, REDUNDANCY AND DATA ANOMALIES

INTRODUCTION

The **Gaming Accessories E-Commerce Management System** is designed to efficiently manage customers, sellers, products, orders, payments, and inventory. Functional dependencies help identify the relationship between attributes in each table, while redundancy analysis and data anomaly detection ensure data consistency and reduce duplication. A well-designed database improves performance, maintains data integrity, and supports efficient online shopping and order management.

1. CUSTOMER TABLE

Functional Dependency

Customer_ID → First_Name, Last_Name, Email, Phone, Address, Password

Redundancy Analysis

Customer information is stored only once in the Customer table. Orders store only the Customer_ID, preventing duplicate customer records.

Data Anomalies

- Insert Anomaly**:** A new customer can be added without placing an order.
- Update Anomaly**:** Updating a customer's phone number requires changes only in the Customer table.
- Delete Anomaly: Deleting an order will not remove customer information.

2. SELLER TABLE

Functional Dependency

Seller_ID → Seller_Name, Email, Phone, Address

Redundancy Analysis

Seller details are stored only once in the Seller table. Products reference the seller using Seller_ID instead of repeating seller information.

Data Anomalies

- Insert Anomaly**:** A new seller can be added without adding products.
- Update Anomaly**:** Updating seller contact details is done only once.
- Delete Anomaly: Deleting a product does not delete seller information.

3. PRODUCT TABLE

Functional Dependency

Product_ID → Product_Name, Brand, Category, Price, Stock, Seller_ID

Redundancy Analysis

Product details are stored only in the Product table. Order records use Product_ID instead of repeating product information.

Data Anomalies

- Insert Anomaly: New products can be added before receiving orders.
- Update Anomaly**:** Product price or stock is updated only once.
- Delete Anomaly: Deleting an order does not remove product information.

4. ORDERS TABLE

Functional Dependency

Order_ID → Customer_ID, Order_Date, Total_Amount, Order_Status

Redundancy Analysis

The Orders table stores only order-related information. Customer and product details are referenced using foreign keys.

Data Anomalies

- Insert Anomaly**:** Orders can be created independently using Customer_ID.
- Update Anomaly**:** Order status can be updated without affecting other tables.
- Delete Anomaly**:** Deleting an order does not remove customer or product data.

5. PAYMENT TABLE

Functional Dependency

Payment_ID → Order_ID, Payment_Method, Payment_Status, Payment_Date, Amount

Redundancy Analysis

Payment information is stored separately from the Orders table to avoid duplicate payment records.

Data Anomalies

- Insert Anomaly**:** Payment details can be recorded after creating an order.
- Update Anomaly**:** Payment status is updated only once.
- Delete Anomaly**:** Deleting payment information does not remove the order.

6. INVENTORY TABLE

Functional Dependency

Inventory_ID → Product_ID, Quantity_Available, Last_Updated

Redundancy Analysis

Inventory details are maintained in a separate table, preventing repeated stock information in the Product table.

Data Anomalies

- Insert Anomaly**:** Inventory records can be created when a new product is added.
- Update Anomaly**:** Stock quantity changes are updated only once.
- Delete Anomaly**:** Deleting an inventory record does not remove product details.

7. REVIEW TABLE

Functional Dependency

Review_ID → Customer_ID, Product_ID, Rating, Comment, Review_Date

Redundancy Analysis

Customer reviews are stored separately from products and customers, avoiding repeated review information.

Data Anomalies

- Insert Anomaly**:** A customer can add a review only after purchasing a product.
- Update Anomaly**:** Editing a review affects only that review record.
- Delete Anomaly**:** Deleting a review does not remove customer or product information.

CONCLUSION

The **Gaming Accessories E-Commerce Management System** is designed using proper functional dependencies and normalized tables. Each entity stores only its relevant data, reducing redundancy and preventing insert, update, and delete anomalies. This design improves data consistency, maintains integrity, and ensures efficient database management for the gaming accessories e-commerce platform.